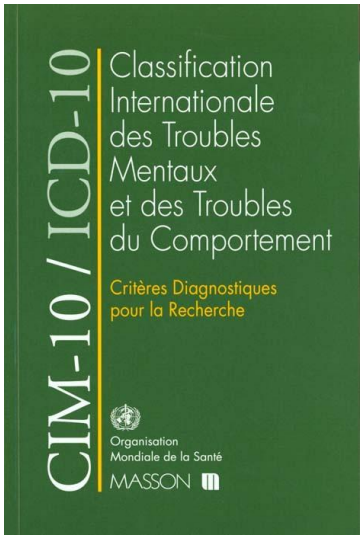
ENSTA Bretagne
2, rue François Verny 29806
BREST cedex FRANCE
Tel +33 (0)2 98 34 88 00
www.ensta-bretagne.fr

Rapport

Projet AIME

Projet AIME



ROMAN DARD, ANDRY CALVEZ, ÉMILIEN QUÉRÉ, HASSAN BAKER, JEAN CAYLUS

Résumé

Le projet AIME vise à automatiser l'attribution des codes CIM-10 à partir de comptes rendus médicaux en utilisant des techniques avancées de traitement du langage naturel. Après une première phase consacrée à la conception des blocs de traitement, cette seconde partie du projet s'est focalisée sur l'intégration complète du système, la mise en place d'une interface utilisateur et la validation fonctionnelle.

Trois modules principaux ont été développés et intégrés : un module d'analyse de texte exploitant la bibliothèque MedKit pour la segmentation, la détection de négation et l'extraction d'entités médicales ; un module de classification basé sur le modèle BioLORD pour la correspondance sémantique avec les descriptions CIM-10 ; et enfin, une interface graphique permettant aux médecins de charger un document et de visualiser les propositions de codage.

Des tests unitaires, d'intégration et de validation fonctionnelle ont été menés pour évaluer la robustesse et la précision du système. Les résultats montrent une capacité du prototype à extraire automatiquement des entités pertinentes et à les associer de façon cohérente à des codes CIM-10, réduisant ainsi la charge administrative des professionnels de santé.

Ce travail illustre la faisabilité d'une solution d'aide au codage médical s'appuyant sur l'intelligence artificielle et ouvre des perspectives concrètes pour un déploiement en contexte hospitalier réel.

Table des matières

Résumé **2**
1. Introduction **6**
2. Etat de l'art **8**
2.1. Définiton des mots clés 8
2.2. Projets existants 8
3. Rétrospective de la première partie du projet **9**
3.1. Contexte initial et problématique 9
3.2. Travaux réalisés et limitations rencontrées 9
4. Architecture détaillée du système final **11**
4.1. Vue d'ensemble technique 11
4.2. Bloc 1 – Analyse de texte (NLP) 12
4.2.1. Objectifs fonctionnels 12
4.2.2. Prétraitement et segmentation 13
4.2.3. Détection des négations 13
4.2.4. Détection des entités médicales 13
4.2.5. Préparation de la sortie du bloc NLP 14
4.3. Bloc de classification CIM-10 14
4.3.1. Objectifs fonctionnels 14
4.3.2. Préparation de la base CIM-10 14
4.3.3. Matching sémantique entre entités et codes 15
4.3.4. Exemple de résultat de matching 16
4.3.5. Robustesse et limites 16
4.4. Bloc 3 – Interface utilisateur 17
4.4.1. Objectifs fonctionnels de l'interface 17
4.4.2. Structure générale du code 17
4.4.3. Perspectives d'amélioration de l'interface 18
5. Démarche d'ingénierie système **19**

5.1. Organisation générale et cadre méthodologique 19
5.2. Planification du projet et diagramme de Gantt 19
5.2.1. Diagramme de Gantt du projet AIME – Acte II 20
5.3. Pratique Agile, outils et suivi de projet 20
5.3.1. Rythme de développement 21
5.3.2. Répartition des rôles dans l'équipe 21
5.4. Autoévaluation et dynamique de groupe 21
5.4.1. Répartition des responsabilités et implication 21
5.4.2. Communication et coordination 22
5.4.3. Gestion des imprévus et adaptabilité 22
5.4.4. Tensions, gestion des désaccords 22
5.4.5. Bilan collectif et progression 23
6. Démarche de tests et validation 24
6.1. Tests unitaires 24
6.1.1. Bloc NLP 24
6.1.2. Bloc Classification CIM-10 24
6.2. Tests d'intégration 26
6.2.1. Intégration 26
6.2.2. Problèmes rencontrés et ajustements 27
6.3. Validation fonctionnelle 28
6.3.1. Scénarios de validation 28
6.3.2. Synthèse VFU (Validation Fonctionnelle Usine) 29
7. Analyse des performances et résultats 30
7.1. Objectifs de l'analyse 30
7.2. Métriques utilisées 30
7.3. Étude de cas – Exemples concrets 30
7.4. Robustesse et limites observées 31
8. Impacts environnementaux et éthiques 32
8.1. Enjeux environnementaux de l'IA médicale 32
8.2. Évaluation qualitative de l'empreinte du projet AIME 32
8.3. Éco-conception logicielle et optimisation 33
8.3.1. Optimisations mises en place 33
8.4. Fin de vie du système et pratiques responsables 34

8.5. Zones d'ombre et améliorations potentielles 35
8.5.1. Limitations actuelles 35
8.5.2. Améliorations possibles à moyen terme 36
9. Perspectives et améliorations futures 37
9.1. Ouvertures techniques à court et moyen terme 37
9.1.1. Enrichissement sémantique des entités médicales 37
9.1.2. Meilleure gestion des temporalités et du contexte 37
9.1.3. Passage à un traitement multilingue 37
9.1.4. Intégration avec des systèmes hospitaliers existants 38
9.2. Perspectives d'industrialisation et déploiement clinique 38
9.2.1. Mise en conformité réglementaire 38
9.2.2. Interface utilisateur avancée et ergonomie clinique 38
9.2.3. Formation, acceptabilité et accompagnement au changement 38
9.2.4. Industrialisation logicielle 39
9.3. Évolutions à long terme 39
10. Conclusion 40
Références 41

1. Introduction

Dans le domaine médical, les résumés d'hospitalisation constituent un support central pour assurer la continuité des soins, retracer le parcours clinique du patient, et garantir la traçabilité des diagnostics. Ces documents, rédigés par les praticiens à l'issue d'un séjour hospitalier, sont également essentiels pour le codage administratif et le remboursement des actes par les systèmes de santé. L'exactitude du codage médical, en particulier celui fondé sur la Classification Internationale des Maladies (CIM-10), joue ainsi un rôle crucial sur le plan clinique mais aussi administratif.

Cependant, l'attribution manuelle de ces codes s'avère souvent longue, fastidieuse et sujette à des erreurs. La complexité du langage médical, les variations dans la rédaction des comptes rendus, la présence de négations ou d'incertitudes, et l'ambiguïté des termes utilisés constituent autant d'obstacles à un codage fiable et homogène. Cette problématique est d'autant plus marquée dans des spécialités telles que la psychiatrie, où les descriptions cliniques sont parfois nuancées et difficilement normalisables.

Face à ces enjeux, le recours à des techniques de traitement automatique du langage (NLP) combinées à l'intelligence artificielle offre une alternative prometteuse. L'objectif du projet AIME (Automatisation de l'Indexation Médicale par l'Extrait) est précisément de concevoir une solution capable d'analyser un compte rendu médical, d'en extraire les informations pertinentes, et de proposer une liste cohérente de codes CIM-10 correspondants. Cette approche vise à alléger la charge administrative des soignants tout en améliorant la fiabilité du codage médical.

Dans une première phase du projet, un prototype fonctionnel avait été mis en place. Il reposait sur une architecture articulée autour de deux blocs principaux : un bloc d'analyse sémantique du texte médical, et un bloc de classification associant les descriptions extraites aux codes CIM-10 correspondants à l'aide d'un modèle de similarité. Les premiers résultats avaient confirmé la faisabilité technique de cette approche, tout en révélant plusieurs défis à relever, notamment en termes de précision, de couverture des cas cliniques, et de robustesse du système.

Cette seconde phase du projet s'inscrit dans une démarche d'aboutissement et d'intégration. Elle a consisté à consolider les modules précédemment développés, à les faire interagir de manière fluide, à concevoir une interface utilisateur simple et efficace, et à mettre en place une méthodologie de test rigoureuse pour valider les performances du système final. Une attention particulière a également été portée à la qualité logicielle, à la maintenabilité du code, ainsi qu'aux conditions réelles d'utilisation dans un contexte hospitalier.

L'ensemble de ces travaux s'est inscrit dans une démarche d'ingénierie système et de développement agile, avec une attention constante portée à la répartition des rôles dans l'équipe, à la planification des tâches (via un diagramme de Gantt), et à la documentation du projet. Le projet AIME Acte II s'est ainsi

voulu à la fois une expérimentation technique poussée, et une mise en situation professionnelle complète, représentative des enjeux actuels de l'intelligence artificielle appliquée à la santé.

Ce rapport rend compte de cette deuxième phase du projet. Il débute par un rappel du contexte et des éléments fondamentaux développés en première partie, avant de détailler l'architecture du système final, les choix techniques réalisés et les modules implémentés. Il présente ensuite la démarche de validation du système, les résultats obtenus, ainsi que les perspectives d'amélioration. Enfin, une analyse critique du fonctionnement de l'équipe et une réflexion sur les impacts éthiques et environnementaux viennent clore ce travail.

2. Etat de l'art

2.1. Définiton des mots clés

Notre projet est lié aux mots clés suivants :

- **Biomedical Engineering**. Discipline qui applique les principes d'ingénierie aux systèmes médicaux et biologiques. Elle combine l'expertise en ingénierie avec les sciences médicales pour développer des technologies innovantes améliorant les soins de santé.
- **Real Data**. Données cliniques authentiques collectées dans un environnement médical réel, par opposition aux données simulées ou synthétiques.
- **Data Processing**. Ensemble des opérations effectuées sur les données brutes pour les transformer en informations exploitables, incluant la collecte, le nettoyage, l'analyse et l'interprétation des données.
- **Unsupervised Classification**. Méthode d'apprentissage automatique qui identifie des patterns et structures dans les données sans étiquettes prédéfinies, permettant de découvrir des groupements naturels dans les données.
- **Text Analysis**. Processus d'extraction d'informations significatives à partir de textes non structurés, utilisant des techniques de traitement du langage naturel et d'analyse sémantique.
- **PMSI** (Programme de Médicalisation des Systèmes d'Information). Système français de description et de mesure de l'activité hospitalière, permettant d'analyser et de valoriser financièrement les séjours hospitaliers.
- **IC D10 (Classification Internationale des Maladies, 10ème révision).** Outil de classification international des maladies et des problèmes de santé connexes, publié par l'OMS, servant de référence pour le codage médical.

2.2. Projets existants

De nombreux projet et applications utilisant l'IA afin de rédiger des comptes-rendus médicaux existent déjà. On pourra notamment citer :

- Nabla Copilot [1]
- Amazon HealthScribe [2]
- Nuance Dragon Ambient eXperience (DAX) [3]

Néanmoins, les projets concernant l'encodage des comptes-rendus selon le CIM-10 sont très rares. Un projet libanais [4] utilisant l'IA dans ce but a retenu notre attention. Nous adapterons ce projet afin qu'il réponde aux contraintes militaires du HIA Clermont-Tonnerre.

3. Rétrospective de la première partie du projet

3.1. Contexte initial et problématique

Au lancement du projet AIME, l'objectif principal était de concevoir une solution capable d'automatiser l'attribution de codes CIM-10 à partir de résumés d'hospitalisation, en s'appuyant sur des techniques de traitement du langage naturel (NLP) et des modèles de similarité sémantique. Ce besoin s'inscrit dans un contexte hospitalier réel, celui du service de psychiatrie du HIA Clermont-Tonnerre, où la diversité des styles de rédaction, l'imprécision de certains termes, et l'importance des nuances contextuelles rendent le codage manuel particulièrement complexe.

Dès les premières semaines, plusieurs problèmes structurels ont été identifiés dans les données et dans la manière d'aborder la tâche :

- Une forte variabilité des comptes rendus selon les praticiens, rendant difficile l'application de règles strictes.
- Une hiérarchie complexe dans la terminologie médicale, où plusieurs formulations pouvaient désigner une même pathologie à des degrés divers.
- La présence fréquente de termes atténués ou hypothétiques, tels que « semble anxieux », « probablement en état de sevrage », difficilement interprétables par des modèles classiques.
- L'usage répandu de la négation, qui inversait la signification médicale d'un terme (ex. « pas de signes de psychose »).
- Des données non supervisées, sans corpus annoté, ce qui excluait l'utilisation directe de modèles supervisés de classification.

Ces constats ont orienté le choix d'une approche modulaire, combinant détection d'entités médicales, détection des négations, et classification par similarité sémantique avec les descriptions du CIM-10.

3.2. Travaux réalisés et limitations rencontrées

Lors de la première phase du projet, plusieurs modules essentiels ont été conçus et testés indépendamment. Le premier travail a consisté à construire une base de référence CIM-10, en extrayant l'ensemble des codes et descriptions disponibles depuis les documents de l'Organisation Mondiale de la Santé (OMS).

Parallèlement, un bloc d'analyse de texte a été développé à l'aide de la bibliothèque MedKit, qui offrait des outils adaptés aux données biomédicales en langue française. Ce bloc incluait :

- Un tokeniseur de phrases et un segmenteur de syntagmes, pour structurer le texte brut.

- Un module de détection des négations, basé sur des règles simples et efficaces.
- Un extracteur d'entités médicales, combinant des règles par expressions régulières (RegexpMatcher) et un modèle de type transformer pré-entraîné sur des corpus biomédicaux (DrBERT-CASM2).

Les entités extraites ont ensuite été transmises à un module de classification, exploitant le modèle BioLORD, un modèle sémantique basé sur Sentence-BERT, spécialisé dans le domaine biomédical. L'algorithme comparait les embeddings vectoriels des entités extraites avec ceux des descriptions CIM-10, en utilisant une mesure de similarité cosinus pour déterminer les meilleures correspondances. Cette approche a permis de produire des premiers résultats encourageants : pour des formulations claires et explicites, le système parvenait à identifier les bons codes avec une bonne précision.

Cependant, plusieurs limitations importantes ont été mises en évidence :

- Le système restait sensible aux variations lexicales et au langage implicite (ex. : « se replie sur lui-même » vs. « trouble anxieux »).
- La détection des entités était parfois trop littérale ou incomplète, notamment dans les longues phrases médicales complexes.
- Le module de similarité proposait plusieurs codes proches mais manquait d'un filtrage contextuel.
- L'absence d'interface utilisateur rendait difficile l'expérimentation complète du système avec un utilisateur final (médecin).
- Enfin, les tests étaient majoritairement manuels et ponctuels, sans stratégie systématique de validation.

Ces constats ont défini les priorités de la seconde phase du projet : renforcer la fiabilité des modules existants, améliorer leur articulation, concevoir une interface, et structurer une véritable démarche de validation logicielle.

4. Architecture détaillée du système final

4.1. Vue d'ensemble technique

À l'issue de la deuxième phase du projet AIME, le système a été consolidé autour d'une architecture modulaire et intégrée, structurée en trois blocs principaux :

- Le bloc d'analyse de texte (NLP), chargé d'extraire les informations médicales pertinentes à partir d'un compte rendu brut.
- Le bloc de classification CIM-10, chargé d'associer les entités extraites à des codes CIM-10 via une analyse sémantique.
- L'interface utilisateur, permettant à un médecin de charger un document, de visualiser les codes proposés et de les valider.

Cette architecture repose sur un flux de données unidirectionnel, avec des modules autonomes comme présenté sur la figure 1. Elle a été pensée pour respecter les principes d'une architecture robuste, maintenable et évolutive, tout en garantissant la simplicité d'usage du point de vue utilisateur.

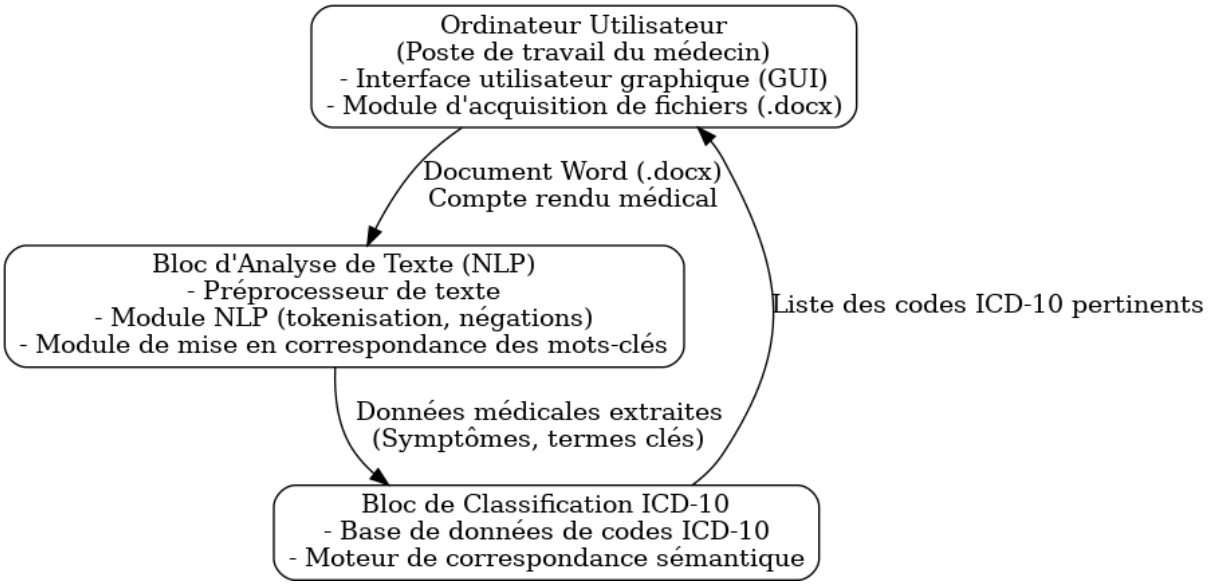


FIGURE 1 - Flux des données dans l'architecture physique

Schéma global de l'architecture :

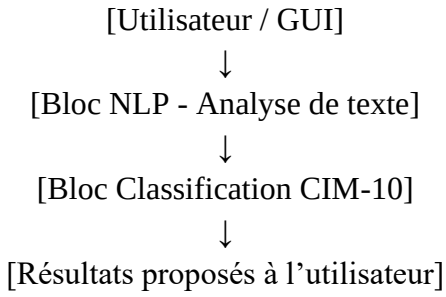


FIGURE 2 - Schéma global de l'architecture

Comme on le comprend sur la figure 2, chaque bloc repose sur des bibliothèques spécialisées ou des modèles pré-entraînés du domaine biomédical. La modularité du système permet d'envisager facilement des évolutions futures, comme le remplacement du moteur de classification, l'ajout de filtres contextuels, ou l'intégration à des systèmes hospitaliers existants.

Dans les sections suivantes, chaque bloc sera décrit en détail, avec ses composants internes, ses choix techniques, les défis rencontrés et les solutions apportées.

4.2. Bloc 1 – Analyse de texte (NLP)

Le premier bloc de l'architecture, dédié à l'analyse du texte médical, constitue une étape cruciale du système. Son rôle est de transformer un document non structuré, rédigé en langage naturel, en une série d'unités d'information exploitables pour la classification. Ce bloc repose sur la bibliothèque MedKit, conçue pour le traitement de textes biomédicaux, et tire parti de modèles linguistiques spécialisés dans le domaine médical.

4.2.1. Objectifs fonctionnels

- Segmenter le texte en phrases puis en syntagmes pour en faciliter l'analyse.
- Identifier les entités médicales (symptômes, pathologies, traitements...) présentes dans le texte.
- Détecter les négations affectant ces entités (ex. « pas de signes de dépression »), afin d'éviter les faux positifs lors de la classification.
- Préparer une sortie standardisée contenant uniquement les entités affirmées et pertinentes, prêtes à être classifiées dans le bloc suivant.

4.2.2. Prétraitement et segmentation

La première étape consiste à lire un document brut (souvent issu d'un .txt ou d'un .pdf médical) à l'aide du module TextDocument de MedKit. Ce texte est ensuite segmenté en phrases à l'aide du SentenceTokenizer, qui s'appuie sur la ponctuation pour identifier les limites des unités de sens. Chaque phrase est ensuite découpée en syntagmes via le SyntagmaTokenizer, selon des séparateurs linguistiques tels que et, mais, etc.

Cette étape permet de travailler sur des fragments de texte plus courts, plus cohérents sémantiquement, et donc plus faciles à analyser.

4.2.3. Détection des négations

Les syntagmes obtenus sont analysés par un module de détection de négation (NegationDetector), utilisant des règles linguistiques précises. Par exemple :

- pas de / sans (avec des exceptions comme sans doute)
- ne semble pas / n'évoque pas

Chaque syntagme est alors annoté avec un attribut booléen indiquant s'il est nié. Cette annotation est transférée aux entités détectées pour les exclure si besoin de la classification finale.

4.2.4. Détection des entités médicales

Une fois les syntagmes segmentés et annotés en fonction de la présence ou non de négations, le système procède à l'extraction des entités médicales à l'aide d'un module de reconnaissance d'entités nommées (NER). Pour cela, nous avons utilisé le composant HFEntityMatcher de MedKit, couplé au modèle DrBERT-CASM2, un transformer pré-entraîné sur des corpus biomédicaux francophones.

Ce modèle est capable d'identifier et de classer des segments de texte selon différentes catégories médicales telles que :

- problem : symptôme ou diagnostic
- treatment : nom de médicament, procédure thérapeutique
- anatomy : parties du corps
- test : examens ou analyses cliniques

Seules les entités classées comme problem (problèmes de santé) sont conservées pour le traitement dans le bloc suivant. L'attribut de négation détecté lors de l'étape précédente est conservé et rattaché à

chaque entité, ce qui permet d'exclure automatiquement les entités qui apparaissent sous une forme niée dans le texte.

Par exemple, dans la phrase : « Le patient ne présente pas de signe de dépression ou d'anxiété aiguë. » le système détectera deux entités (dépression, anxiété aiguë), toutes deux annotées comme niées, et donc non transmises à l'étape de classification.

4.2.5. Préparation de la sortie du bloc NLP

Les entités retenues (affirmées, non négatives) sont ensuite converties dans un format standard : une liste de chaînes de caractères représentant les concepts médicaux clés extraits du document. Ces descriptions médicales, souvent courtes (1 à 5 mots), sont prêtes à être encodées sémantiquement et comparées aux descriptions issues de la base CIM-10.

Un exemple de sortie typique du bloc NLP serait :
["trouble de l'humeur", "hallucinations auditives", "consommation régulière de cannabis", "tentative de suicide ancienne"]

Ces résultats sont transmis au bloc de classification CIM-10, qui sera détaillé dans la section suivante. Ce découplage clair entre l'extraction et la classification permet de maintenir une grande flexibilité dans l'évolution du système.

4.3. Bloc de classification CIM-10

Le second bloc du système AIME est responsable de l'attribution automatique des codes CIM-10 aux descriptions médicales extraites du texte. Il repose sur une approche sémantique non supervisée, qui permet de comparer des fragments de texte libres à des définitions issues de la classification officielle de l'OMS, sans nécessiter de jeu de données annoté au préalable.

4.3.1. Objectifs fonctionnels

- Associer chaque entité médicale détectée (affirmée) à un ou plusieurs codes CIM-10 pertinents.
- S'appuyer sur la similarité sémantique entre les descriptions issues du texte clinique et celles présentes dans la base CIM-10.
- Fournir un classement des codes les plus proches pour chaque entité, en laissant à l'utilisateur la possibilité de valider ou corriger les suggestions.

4.3.2. Préparation de la base CIM-10

La première étape a consisté à créer une base de données des codes CIM-10, contenant pour chaque entrée :

- Le code (ex. : F32.1)
- Sa description officielle (ex. : Épisode dépressif modéré)

Cette base a été nettoyée et restreinte à un sous-ensemble pertinent pour la psychiatrie, dans le but d'améliorer la précision des résultats et de réduire les interférences sémantiques avec d'autres spécialités médicales.

Chaque description a été encodée à l'aide du modèle BioLORD, un Sentence-BERT préentraîné spécifiquement sur des données biomédicales. Le modèle transforme chaque description en un vecteur denses de 768 dimensions, représentant sa signification sémantique dans l'espace vectoriel.
Ces vecteurs sont conservés en mémoire et servent de référentiel pour la comparaison avec les entités extraites.

4.3.3. Matching sémantique entre entités et codes

Une fois les entités médicales extraites par le bloc NLP, celles-ci sont elles aussi encodées en vecteurs sémantiques à l'aide du même modèle BioLORD. Cela garantit la cohérence entre l'espace vectoriel des descriptions issues du texte médical et celui des définitions CIM-10.

Chaque entité extraite (ex. : trouble anxieux, abus de cannabis) est donc transformée en un vecteur, puis comparée à tous les vecteurs représentant les descriptions CIM-10. La mesure de similarité cosinus est utilisée pour évaluer la proximité sémantique entre deux vecteurs :

$$\text{similarité}(A, B) = \frac{A \cdot B}{|A| \cdot |B|}$$

FORMULE 1 - Formule de similarité cosinus

Cette opération est effectuée pour chaque entité, ce qui permet d'obtenir un classement décroissant des codes CIM-10 les plus proches, avec leurs scores de similarité respectifs. Le système conserve par défaut les trois meilleures correspondances afin de proposer plusieurs choix pertinents à l'utilisateur final.

4.3.4. Exemple de résultat de matching

Entité extraite	**Consommation régulière de cannabis**	
Résultat proposé	**Code CIM-10**	**Score**
---	---	
Usage nocif de cannabinoïdes	F12.1	0.89
Intoxication aiguë aux cannabinoïdes	F12.0	0.85
Problèmes liés à l'usage de drogue	Z72.2	0.82

FIGURE 3 - Exemple de résultat de Matching

Dans cet exemple, l'utilisateur peut confirmer le premier code, ou bien sélectionner une alternative en fonction du contexte clinique.

4.3.5. Robustesse et limites

Cette approche présente plusieurs avantages :

- Elle s'adapte bien à des formulations variées, même lorsque les termes exacts du CIM-10 ne sont pas utilisés.
- Elle ne nécessite pas d'apprentissage supervisé, ce qui la rend robuste aux changements de contexte ou aux nouveaux corpus.
- Elle permet de conserver la transparence du raisonnement, en affichant les correspondances proposées et leurs scores.

Néanmoins, certaines limites ont été observées :

- Les scores de similarité proches peuvent rendre le classement délicat (ex. : plusieurs codes proches pour une même entité).
- L'approche n'intègre pas encore de raisonnement contextuel : chaque entité est traitée indépendamment, sans prendre en compte le reste du document.

Ces limites ont été partiellement compensées par des ajustements manuels de la base CIM-10 (suppression de doublons, regroupement de synonymes) et pourraient faire l'objet d'améliorations dans une version future du système (cf. chapitre 8). Certains cas ambigus nécessiteraient une validation humaine systématique pour garantir la précision du codage final.

4.4. Bloc 3 – Interface utilisateur

Afin de rendre le système accessible aux utilisateurs finaux, notamment aux professionnels de santé, une interface graphique intuitive a été développée à l'aide de la bibliothèque PyQt5. Cette interface constitue le point d'entrée du système : elle permet à l'utilisateur de saisir un texte médical libre (compte rendu, note clinique, etc.), de lancer l'analyse, et de visualiser les codes CIM-10 proposés avec leurs descriptions.

4.4.1. Objectifs fonctionnels de l'interface

- L'interface a été pensée pour répondre aux besoins suivants :
- Permettre la saisie directe ou le copier-coller d'un texte médical brut.
- Proposer une interaction minimale mais fonctionnelle, avec des boutons d'action.
- Afficher les résultats de manière structurée, sous forme de tableau, avec les codes, descriptions et entités de référence détectées.
- Intégrer un comportement le plus réactif possible

4.4.2. Structure générale du code

Le fichier principal main.py contient la classe AIME, héritée de QMainWindow, qui instancie l'interface et gère ses interactions. Trois écrans sont intégrés à l'aide d'un widget empilé (stackedWidget) :

- Accueil et zone de saisie du texte
- Options ou paramètres (si présents)
- Écran de chargement et affichage des résultats

Le bouton Valider (pushButton_valider) déclenche l'analyse du texte saisi. Le processus se déroule en plusieurs étapes :

- Extraction du texte pertinent via regex.extract_text().
- Détection des pathologies via pathologies.extract_pathologies().
- Appariement avec la CIM-10 via matcher.match_cim10().

Chaque entité détectée est comparée à la base de données CIM-10, et les meilleurs résultats sont affichés dynamiquement dans un tableau (QTableView) via un QStandardItemModel.

Les colonnes du tableau sont les suivantes :

- Code CIM-10
- Description

- Priorité (fixée par défaut à « haute », mais évolutive)
- Référence à (entité médicale extraite du texte initial)

4.4.3. Perspectives d'amélioration de l'interface

Bien que l'interface actuelle remplisse les fonctions de base attendues, plusieurs améliorations pourraient être envisagées pour renforcer l'efficacité du système auprès des utilisateurs. L'ajout de la possibilité de charger directement des fichiers médicaux (PDF, .docx), par exemple, permettrait d'éviter la saisie manuelle des informations, en utilisant des bibliothèques comme pdfplumber ou python-docx pour une extraction automatique du contenu. En outre, l'intégration de fonctionnalités de filtrage et de tri des résultats, par code, entité extraite, ou pertinence, améliorerait la capacité des médecins à prioriser les diagnostics les plus probables grâce à un score de similarité sémantique. Enfin, un soin particulier apporté à l'esthétique de l'interface, avec des thèmes sombres, des couleurs codées par type de pathologie et des icônes intuitives, rendrait l'outil plus lisible, professionnel et agréable à utiliser.

5. Démarche d'ingénierie système

5.1. Organisation générale et cadre méthodologique

La deuxième phase du projet AIME s'est déroulée dans une logique de développement itératif, combinant les principes de l'ingénierie système et d'une méthodologie agile adaptée à un contexte académique. L'objectif était de structurer le projet autour de jalons clairs (objectifs techniques et livrables), tout en restant flexible face aux aléas rencontrés durant le développement.

La démarche s'est articulée autour de quatre axes principaux :

- La définition précise des sous-systèmes (analyse NLP, classification, interface utilisateur), avec une identification claire des entrées, sorties et points d'interconnexion.
- La gestion des exigences liées à l'usage du système par des praticiens, à la pertinence des résultats, à la maintenabilité du code et à la compatibilité des modules.
- L'intégration progressive des blocs fonctionnels, en conservant une modularité forte pour faciliter les tests et l'évolution du système.
- La validation fonctionnelle basée sur des scénarios réalistes d'utilisation médicale et une traçabilité des résultats obtenus.

Le cadre méthodologique s'est appuyé sur des outils concrets :

- Un diagramme de Gantt établi en début de phase (voir section 4.2), mis à jour régulièrement.
- Des réunions hebdomadaires d'équipe, avec revue de l'avancement, priorisation des tâches, et ajustements en fonction des retours.
- Des documents de suivi partagés (Google Docs), utilisés pour répertorier les bugs, hypothèses, idées d'amélioration ou validations.

Cette approche a permis de maintenir une cohérence globale du projet, tout en laissant à chaque membre de l'équipe la liberté de s'approprier les outils ou langages correspondant à son expertise.

5.2. Planification du projet et diagramme de Gantt

La planification du projet a été un élément central de notre organisation. Elle s'est concrétisée dès le lancement de la seconde phase par l'élaboration d'un diagramme de Gantt, permettant de structurer les différentes étapes de conception, de développement, d'intégration, de tests et de production des livrables.

Le diagramme ci-dessous présente la répartition des tâches sur l'ensemble du semestre, en s'appuyant sur les jalons imposés par l'UE4.4 ainsi que nos propres objectifs d'équipe :

5.2.1. Diagramme de Gantt du projet AIME – Acte II

Phase	Début	Fin
Réunion de lancement	14/01/2025	14/01/2025
Élaboration du Gantt	14/01/2025	21/01/2025
Validation des objectifs	21/01/2025	21/01/2025
Finalisation du bloc NLP	22/01/2025	04/02/2025
Finalisation du bloc CIM-10	22/01/2025	04/02/2025
Tests préliminaires de chaque bloc	05/02/2025	27/02/2025
Intégration des blocs	28/02/2025	14/03/2025
Tests d'intégration	17/03/2025	24/03/2025
Préparation de la démonstration	25/03/2025	28/04/2025
Rédaction du rapport	25/03/2025	16/04/2025
Préparation et finalisation du poster	17/04/2025	24/04/2025
Démonstration devant le jury	29/04/2025	29/04/2025

FIGURE 4 - Diagramme de Gantt du projet AIME - ACTE II

Ce diagramme nous a servi de référentiel collectif tout au long du projet. Chaque sous-phase était associée à des livrables partiels ou à des objectifs vérifiables, permettant de suivre l'avancement avec clarté. En février, les blocs NLP et CIM-10 ont été consolidés individuellement, avec des tests indépendants. En mars, l'effort s'est porté sur l'intégration et la résolution des problèmes d'interopérabilité. En avril, la priorité a été donnée à la stabilisation du système, à la production des livrables finaux (rapport, poster), et à la préparation de la démonstration.

L'ensemble de la planification a été réévalué régulièrement lors des points d'équipe, afin de garder une certaine agilité sans compromettre les délais fixés par l'UE.

5.3. Pratique Agile, outils et suivi de projet

Afin de mener à bien un projet aussi modulaire et évolutif que AIME, nous avons adopté une approche de travail inspirée des méthodes Agile, plus particulièrement du mode Scrum allégé. Cette méthode nous a permis d'assurer une avancée incrémentale, en conservant une souplesse dans la répartition des tâches.

5.3.1. Rythme de développement

Le travail a été découpé en cycles hebdomadaires, chaque semaine faisant l'objet :
d'un point de lancement en début de séance (bilan des tâches réalisées, identification des points de blocage), d'une répartition actualisée des tâches, selon les avancées ou imprévus, et d'un mini-retour d'expérience à la fin de la séance, permettant d'ajuster les priorités de la semaine suivante.

Cette organisation a permis de renforcer l'autonomie de chaque binôme, tout en gardant une cohérence d'ensemble, mais aussi de détecter rapidement les problèmes d'intégration ou de dépendances entre modules et finalement de capitaliser sur les essais/erreurs de chaque bloc en les partageant avec l'ensemble de l'équipe.

5.3.2. Répartition des rôles dans l'équipe

Même si l'ensemble des membres a participé aux grandes étapes du projet, certains rôles se sont naturellement répartis selon les préférences. Un binôme était principalement chargé du bloc NLP, en particulier du prétraitement, de la détection d'entités et des tests liés à MedKit. Un autre s'est concentré sur le modèle de classification sémantique, avec l'optimisation de BioLORD et la gestion de la base CIM-10.

Nous nous sommes ensuite tous impliqués dans la coordination globale, l'intégration des modules, la planification et la rédaction du rapport. Ce découpage n'a cependant jamais été rigide : des échanges fréquents ont permis à chacun de monter en compétence sur d'autres blocs, garantissant une vision d'ensemble du projet.

5.4. Autoévaluation et dynamique de groupe

Le projet a également constitué un exercice de gestion collective, où la communication et l'adaptabilité de chacun ont joué un rôle déterminant dans la progression travail. Nous avons pris le temps de réfléchir à notre fonctionnement d'équipe, d'identifier les leviers de notre efficacité, ainsi que les difficultés rencontrées.

5.4.1. Répartition des responsabilités et implication

L'un des atouts de notre équipe a été une répartition relativement équilibrée des responsabilités : chacun a su trouver sa place selon ses affinités techniques, tout en acceptant d'intervenir ponctuellement sur d'autres volets lorsque le besoin s'en faisait sentir. Cette capacité à sortir de sa zone de confort a été essentielle, notamment lors des phases d'intégration ou de tests, qui nécessitaient une compréhension transversale de tous les blocs.

Chaque membre s'est activement impliqué dans le projet, avec une régularité dans le travail personnel en dehors des séances encadrées. Des efforts particuliers ont été faits sur :

- La documentation du code, pour faciliter les transitions entre modules,
- La rédaction collaborative, notamment lors de la production du rapport final,
- L'échange de compétences, que ce soit sur l'usage de MedKit ou la gestion des modèles NLP.

5.4.2. Communication et coordination

Dans l'ensemble, la communication au sein du groupe a été fluide et bienveillante, grâce à l'utilisation régulière d'outils de messagerie (WhatsApp, Messenger) et de documents partagés. Les réunions hebdomadaires ont constitué des points d'ancrage importants, permettant de faire le point sur les avancées mais aussi d'exprimer librement les blocages techniques ou organisationnels et donc de redéfinir les priorités en fonction des jalons à venir.

5.4.3. Gestion des imprévus et adaptabilité

Comme dans tout projet technique, plusieurs aléas ont nécessité des ajustements en cours de route : L'intégration des blocs a pris plus de temps que prévu, notamment à cause de problèmes de compatibilité entre les formats de sortie et d'entrée des modules. Certains choix techniques initiaux (par exemple, l'utilisation d'un modèle trop lourd pour certains postes) ont dû être reconsidérés rapidement, en adaptant les solutions à nos contraintes matérielles.

Face à ces imprévus, l'équipe a su faire preuve d'agilité : redéfinition rapide des objectifs à court terme, réaffectation des tâches, simplification de certaines fonctionnalités pour respecter les délais… L'entraide entre membres a été déterminante pour éviter les blocages prolongés, notamment lorsqu'un binôme rencontrait des difficultés techniques sur un module donné.

5.4.4. Tensions, gestion des désaccords

Le projet s'est globalement déroulé dans une ambiance constructive. Quelques désaccords ponctuels ont émergé, principalement autour :

- Du choix des outils de traitement,
- De la priorité à donner à certaines fonctionnalités (ex. : enrichissement de la base CIM-10 vs. interface utilisateur).

Ces situations ont été résolues par la discussion, en se recentrant sur les objectifs techniques et les contraintes de temps. La capacité de l'équipe à écouter les arguments de chacun, sans posture « d'autorité », a évité que ces désaccords ne se transforment en conflits.

5.4.5. Bilan collectif et progression

Avec du recul, cette deuxième phase du projet a été l'occasion pour l'ensemble du groupe :

- de renforcer sa capacité à travailler en équipe, dans un cadre semi-autonome,
- de gagner en rigueur, notamment en matière de documentation, d'intégration et de validation,
- de mieux comprendre l'importance d'une communication régulière et structurée dans la réussite d'un projet collectif.

Enfin, le projet AIME nous a permis de vivre une expérience proche d'un environnement professionnel réel, avec des responsabilités réparties, des livrables à produire, et une logique de démonstration publique à la fin du projet.

6. Démarche de tests et validation

Dans le cadre du projet AIME, la phase de tests a constitué une étape essentielle pour garantir la fiabilité, la robustesse et la cohérence de l'ensemble du système. Étant donné l'architecture modulaire adoptée, les tests ont été réalisés selon une approche progressive, incluant :

- Des tests unitaires sur les blocs individuels,
- Des tests d'intégration lors de la mise en relation des modules entre eux,
- Une validation fonctionnelle fondée sur des scénarios réalistes et proches de l'utilisation finale.

Cette démarche s'est appuyée sur des cas cliniques simulés, des jeux de données test issus de résumés d'hospitalisation psychiatrique, et une vérification manuelle des résultats. L'objectif était non seulement de s'assurer du bon fonctionnement du système, mais aussi de qualifier la pertinence des résultats médicaux proposés.

6.1. Tests unitaires

Les tests unitaires ont été réalisés indépendamment sur chacun des blocs fonctionnels (analyse NLP, classification CIM-10, interface utilisateur) afin de valider la bonne exécution des fonctions clés. Chaque test visait à isoler une fonctionnalité pour vérifier que, dans un contexte maîtrisé, elle produit les résultats attendus.

6.1.1. Bloc NLP

Des tests ont été menés sur :

- La segmentation en syntagmes, pour vérifier que le SyntagmaTokenizer de MedKit produisait bien les découpages attendus à partir de phrases complexes.
- La détection de négation, via des syntagmes contenant ou non des structures comme "pas de", "ne semble pas", ou "sans doute". L'objectif était de valider que l'attribut is_negated était bien associé et transféré correctement aux entités.
- L'extraction d'entités médicales, pour vérifier que le modèle HFEntityMatcher identifiait correctement les termes cliniques dans un échantillon de phrases psychiatriques types (ex. : troubles anxieux, idées suicidaires, hallucinations auditives).

Chaque test s'est appuyé sur des phrases contrôlées, dont les entités attendues étaient connues à l'avance.

6.1.2. Bloc Classification CIM-10

Le cœur des tests de ce bloc visait à vérifier la cohérence de la similarité sémantique utilisant le modèle BioLORD. Plusieurs éléments ont été testés de manière isolée :
Encodage des entités médicales

Chaque entité extraite devait être encodée correctement en vecteurs via Sentence-BERT (BioLORD). Des tests ont vérifié que des phrases similaires (ex. : trouble de l'humeur vs humeur dépressive) donnaient des vecteurs proches.

Calcul de similarité cosinus

Le calcul de la similarité entre entités encodées et descriptions CIM-10 a été validé mathématiquement, avec des cas extrêmes :

- similarité proche de 1 pour des correspondances parfaites ;
- similarité faible (< 0.5) pour des entités sans lien clinique direct.

Ordonnancement des résultats

Le système devait retourner une liste triée des meilleurs codes pour une entité donnée. Des assertions ont été mises en place pour vérifier que :

- le meilleur score correspondait bien au code attendu,
- aucun code incohérent (ex. : relevant de la cardiologie pour une entité psychiatrique) ne remontait dans les premiers résultats.

Cas test représentatif

Entrée : "trouble anxieux généralisé"

Attendu : code F41.1 (trouble anxieux généralisé) dans les 2 premiers résultats.

Résultat : F41.1 (score = 0.94), F41.0 (score = 0.89)

Test réussi

Bloc Interface Utilisateur

Bien que l'interface repose en grande partie sur des interactions visuelles, plusieurs tests unitaires ont pu être réalisés sur les fonctions appelées lors des actions utilisateur, notamment :

Chargement du texte saisi

La fonction valider_clicked() a été testée avec divers contenus saisis dans la plainTextEdit. Des assertions ont permis de vérifier que le texte était bien transmis aux modules de traitement.

Transmission et affichage des résultats

Une fois les codes CIM-10 obtenus, des tests ont vérifié que chaque ligne contenait bien :

- un code,
- une description cohérente,
- une entité source,
- une priorité (par défaut : “haute”).

Robustesse face à des cas vides ou malformés

Le système a été testé avec :

- un champ vide (le bouton "Valider" ne devait pas déclencher d’erreur),
- des textes non médicaux ou incomplets (vérification que la table restait vide sans plantage).

L’ensemble de ces tests unitaires a permis de valider que chaque bloc fonctionnait de manière isolée, selon les attentes.

6.2. Tests d’intégration

Une fois les modules individuels validés par les tests unitaires, une attention particulière a été portée à leur intégration au sein du système complet. L’objectif de cette phase était de vérifier que :

- les blocs communiquaient correctement entre eux (formats, encodage, ordre d’exécution),
- les résultats produits en bout de chaîne étaient cohérents,
- aucune erreur ne survenait lors d’un enchaînement d’actions utilisateur.

L’intégration a été réalisée progressivement, en commençant par le chaînage NLP → classification, puis en ajoutant l’interface utilisateur.

6.2.1. Intégration

Intégration NLP + Classification CIM-10

Le premier test d’intégration a consisté à valider que les entités extraites par le module NLP (non niées) pouvaient être transmises sans transformation supplémentaire au module de classification.

Pour cela :

- Une entité type (ex. : hallucinations auditives) a été injectée dans le pipeline complet.

- Le résultat du matching a été vérifié manuellement, en s'assurant de la cohérence clinique des codes proposés (F23.1 – Trouble psychotique aigu).

Des vérifications ont été faites sur :

- La chaîne d'encodage UTF-8, pour éviter les erreurs sur les accents ou les caractères spéciaux.
- L'absence d'entités vides ou mal filtrées (notamment les entités niées, qui doivent être ignorées).

Intégration complète avec interface

Dans un second temps, le bloc interface a été connecté au pipeline NLP + classification. Le test visait à simuler un scénario complet :

- Saisie d'un texte médical par l'utilisateur.
- Appui sur le bouton "Valider".
- Analyse du texte → extraction → classification.
- Affichage des résultats dans la table.

Exemple de scénario intégré testé :

Texte saisi : "Le patient présente des idées suicidaires, un comportement d'isolement et un usage chronique de cannabis. Pas de signes de délire."

Résultat : trois entités affirmées transmises au matching.

Codes CIM-10 proposés :

- F12 – Troubles mentaux et du comportement liés à l'utilisation de dérivés du cannabis
- Z86.4 – Antécédents personnels d'abus de substances psychoactives
- F32.1 – Épisode dépressif modéré

L'entité niée (délire) a été correctement exclue.

6.2.2. Problèmes rencontrés et ajustements

Quelques problèmes d'intégration ont nécessité des corrections ponctuelles :

- Des entités contenant des guillemets ou caractères spéciaux provoquaient des erreurs à l'étape de vectorisation → ajout d'un nettoyage des chaînes.

- Le tableau affichait parfois des résultats en double → déduplication ajoutée en amont dans la construction de self.codes_finaux.
- En cas de texte très court ou très long, des comportements imprévisibles apparaissaient → ajout de messages d'erreur simples (ex. : "aucune entité détectée").

Ces tests d'intégration ont permis de valider que l'ensemble du système fonctionnait de manière fluide, de la saisie utilisateur à l'affichage final, sans intervention manuelle entre les blocs.

6.3. Validation fonctionnelle

La validation fonctionnelle constitue la dernière étape de vérification du système, en s'appuyant sur des scénarios cliniques réalistes. Elle vise à répondre à la question suivante :
« Le système permet-il effectivement à un médecin de saisir un texte et d'obtenir une suggestion cohérente de codage CIM-10, exploitable en pratique ? »

Cette phase a été pensée avec un regard extérieur, simulant le rôle d'un praticien. Elle s'appuie sur une logique de chaînes fonctionnelles, c'est-à-dire des enchaînements d'actions qui doivent produire un résultat observable et conforme aux attentes.

6.3.1. Scénarios de validation

Plusieurs cas d'usage ont été définis à partir de comptes rendus psychiatriques simplifiés (anonymisés). Chaque scénario incluait :

- Un texte saisi (ou copié dans l'interface),
- Les entités attendues,
- Les codes CIM-10 espérés (manuellement déterminés),
- Une vérification des résultats proposés.

Scénario 1 : épisode dépressif simple

Texte saisi : "Le patient présente une humeur triste, des troubles du sommeil et des idées de dévalorisation, sans idée suicidaire à ce jour."

Entités attendues : humeur triste, troubles du sommeil, idées de dévalorisation.

Entité niée : idée suicidaire.

Résultats obtenus :

F32.9 – Épisode dépressif, sans précision
F39 – Trouble de l'humeur [affectif], sans précision\n
Aucun code pour les idées suicidaires (correctement filtré)

Conformité clinique validée

Scénario 2 : consommation de substances et hallucinations

Texte saisi : "Hospitalisé pour usage chronique de cannabis, hallucinations auditives, agitation nocturne et repli sur soi. Pas de symptômes de manie."

Entités détectées : usage de cannabis, hallucinations auditives, agitation, repli sur soi.

Résultats obtenus :
F12 – Troubles mentaux et du comportement liés à l'utilisation de dérivés du cannabis
F16 – Troubles mentaux et du comportement liés à l'utilisation d'hallucinogènes
F51.9 – Trouble du sommeil non organique, sans précision

Résultat jugé très satisfaisant par rapport au contexte

6.3.2. Synthèse VFU (Validation Fonctionnelle Usine)

La validation fonctionnelle a permis de notifier :

Les écarts constatés, par exemple :

- une entité oubliée dans un texte long,
- une confusion entre deux codes proches (F41.1 vs F41.0),

Les actions correctives envisagées :

- ajout de logs de traitement,
- messages d'erreur plus clairs,
- amélioration du nettoyage des entités.

Conclusion de la validation

L'ensemble des scénarios testés a permis de confirmer que :
Le système produit des résultats globalement fiables et pertinents,
Il tient compte de la négation et ignore les entités niées,
Il offre une base de travail solide pour le praticien, même si une validation humaine reste nécessaire.
Le prototype est ainsi considéré comme fonctionnel dans son usage cible, et prêt à être amélioré pour une intégration dans un environnement réel plus exigeant.

7. Analyse des performances et résultats

7.1. Objectifs de l'analyse

La validation fonctionnelle a permis de vérifier que le système fonctionne correctement dans des cas cliniques réalistes. Toutefois, pour évaluer objectivement la qualité du système, il est nécessaire de mesurer ses performances sur des critères quantifiables.

Cette analyse vise à répondre à plusieurs questions clés :

- Quelle est la précision du système dans la suggestion des bons codes CIM-10 ?
- Quels sont les temps de réponse pour une analyse complète, du texte saisi jusqu'à l'affichage final ?
- Le système est-il robuste face à la variabilité des formulations, à des phrases longues, ou à des données bruitées ?
- Quels sont les cas d'erreurs fréquents (faux positifs, faux négatifs) et leurs causes ?

L'évaluation repose sur une série de tests systématiques effectués à partir :

- d'un jeu de 10 textes médicaux psychiatriques, anonymisés et synthétisés à partir de vrais cas,
- d'une vérité terrain définie manuellement (codes attendus pour chaque texte),
- et d'un protocole standardisé, avec comparaison des résultats proposés par le système à ceux déterminés a priori.

7.2. Métriques utilisées

Plusieurs indicateurs classiques en classification ont été utilisés pour quantifier la qualité du système :

- Exactitude Top-1 : pourcentage de cas où le premier code CIM-10 proposé par le système est correct.
- Exactitude Top-3 : pourcentage de cas où le bon code figure parmi les trois premiers proposés.
- Faux positifs : cas où le système propose un code sans lien clinique avec l'entité détectée.
- Faux négatifs : cas où une entité valide n'est associée à aucun code pertinent.
- Temps moyen d'analyse : durée entre le clic sur "Valider" et l'apparition des résultats.

7.3. Étude de cas – Exemples concrets

Cas réussi (Top-1 correct)

Texte : "Le patient est hospitalisé pour un épisode dépressif majeur avec idées noires."
Entité : épisode dépressif majeur

Résultat proposé : F32.2 – Épisode dépressif sévère sans symptômes psychotiques
Commentaires : Le système a correctement identifié la sévérité.

Cas Top-3 correct mais Top-1 incorrect

Texte : "Isolement social marqué et perte de l'estime de soi."
Résultat proposé (Top-1) : Z60.2 – Difficultés liées à la socialisation
Référence correcte (Top-2) : F32.0 – Épisode dépressif léger
Commentaires : Le système a hésité entre un facteur environnemental (Z60) et un diagnostic psychiatrique (F32), sans contexte clinique plus précis.

Cas d'échec (faux négatif)

Texte : "Patient avec antécédents d'hallucinations auditives dans un contexte de sevrage."
Résultat : Aucun code pertinent proposé
Commentaires : Le mot "antécédents" a sans doute été mal interprété, et la phrase n'a pas été retenue comme affirmée → nécessité d'améliorer le traitement du contexte temporel.

7.4. Robustesse et limites observées

Le système est globalement robuste face aux variations lexicales : des synonymes ou paraphrases simples sont correctement encodés. Il est sensible au contexte négatif ou hypothétique, ce qui est volontairement géré par l'analyseur de négation.

Il reste perfectible sur :

- la gestion des antécédents ou temporalités ("ancien", "passé", "récidive", etc.),
- les entités ambiguës ou implicites (ex. : "se replie sur lui-même"),
- la différenciation facteurs sociaux vs diagnostics cliniques, notamment pour les codes Z.

8. Impacts environnementaux et éthiques

8.1. Enjeux environnementaux de l'IA médicale

Le développement de solutions d'intelligence artificielle dans le domaine médical soulève des questions environnementales importantes. En particulier, les systèmes basés sur des modèles de langage ou des architectures de type transformer, comme c'est le cas dans le projet AIME, peuvent présenter une empreinte énergétique non négligeable.

Les principaux facteurs d'impact environnemental identifiés sont les suivants :

- Le coût énergétique de l'inférence, notamment lors de l'utilisation de modèles comme BioLORD ou DrBERT.
- La consommation liée au stockage et à la manipulation de la base CIM-10, surtout si celle-ci est enrichie avec des synonymes ou des vecteurs de grande taille.
- La charge de calcul locale, qui dépend du poste utilisateur (CPU ou GPU) et peut varier fortement selon les capacités de la machine.
- Le risque d'externalisation non maîtrisée si certaines fonctions sont hébergées sur des serveurs distants (cloud), avec une traçabilité énergétique incertaine.

8.2. Évaluation qualitative de l'empreinte du projet AIME

Le projet AIME repose sur une architecture légère et locale, dans laquelle :

- les modèles sont chargés en mémoire à la demande, et leur utilisation reste ponctuelle,
- aucun entraînement de modèle n'est effectué : seuls des modèles pré-entraînés sont utilisés pour l'inférence,
- les bases de données sont restreintes à un sous-ensemble ciblé (psychiatrie uniquement), ce qui limite les ressources nécessaires.

Ainsi, même si l'utilisation de modèles pré-entraînés comme BioLORD implique une certaine consommation, l'empreinte du projet dans sa forme actuelle (prototype étudiant) reste modérée.

En revanche, une éventuelle industrialisation ou généralisation du système à d'autres spécialités impliquerait une réflexion plus poussée sur la sobriété numérique et l'optimisation des traitements, notamment pour les établissements hospitaliers peu équipés.

8.3. Éco-conception logicielle et optimisation

Conscients des enjeux environnementaux liés au numérique, nous avons intégré dès la conception du projet AIME des principes d'éco-conception logicielle, visant à optimiser les performances tout en réduisant l'empreinte énergétique du système. Cette approche ne se limite pas aux modèles utilisés, mais s'applique à l'ensemble de la chaîne logicielle.

8.3.1. Optimisations mises en place

Restriction thématique de la base CIM-10

Afin de limiter les traitements inutiles et les comparaisons sémantiques sur des entrées non pertinentes, nous avons volontairement réduit la base CIM-10 aux pathologies psychiatriques. Cette réduction permet :

- d'alléger les temps de traitement,
- de réduire la taille des vecteurs à stocker en mémoire,
- d'améliorer la précision tout en diminuant la charge machine.

Limitation du nombre de codes retournés

Le moteur de correspondance sémantique retourne uniquement les 5 meilleures suggestions (Top-5) pour chaque entité extraite. Cela évite :

- des affichages inutiles dans l'interface,
- des comparaisons vectorielles superflues,
- une surcharge cognitive pour l'utilisateur.

Encodage intelligent des entités

Lors de l'analyse d'un texte, chaque entité est encodée une seule fois, et les résultats sont mémorisés dans une structure temporaire. Si la même entité revient dans un autre document, le système peut à terme envisager une mise en cache locale pour éviter un nouveau passage par le modèle.

Gestion mémoire sobre

Les modules de classification et d'affichage sont désactivés en dehors des appels utilisateurs (par exemple, pas d'analyse continue ou en arrière-plan). Cela réduit :

- la consommation processeur,
- les accès disque fréquents,
- la chauffe inutile du poste utilisateur.

Perspectives d'optimisation futures

Mise en cache des embeddings CIM-10 sur disque pour éviter un recalcul à chaque exécution.
Découplage interface / moteur IA, afin de pouvoir déporter les calculs lourds vers des machines dédiées plus sobres en énergie (ex. : serveurs optimisés, cloud vert).
Utilisation d'alternatives légères aux modèles transformers, par exemple des modèles à base de TF-IDF ou fastText pour certaines tâches simples (préfiltrage).

Ces pistes sont d'autant plus pertinentes dans un contexte hospitalier réel, où le déploiement sur des postes utilisateurs variés (ordinateurs de consultation, dossiers patients informatisés) impose une efficacité et une légèreté maximales.

8.4. Fin de vie du système et pratiques responsables

Un projet logiciel responsable ne se limite pas à sa phase de développement et d'usage actif : il doit aussi anticiper sa durée de vie, sa maintenabilité, et les bonnes pratiques liées à sa fin de cycle. Dans le cas du projet AIME, plusieurs éléments ont été pensés pour favoriser une approche durable et éthique du système, même au-delà de son utilisation immédiate.

Simplicité de désinstallation et nettoyage

Le système ne dépend pas de composants systèmes invasifs (pas de services système, pas d'écriture dans le registre). Tous les fichiers liés (modèles, bases, modules) peuvent être désinstallés manuellement ou supprimés en quelques clics, sans laisser de fichiers temporaires persistants. Aucun cloud externe ou serveur distant n'est sollicité, ce qui limite les externalités invisibles.

Traçabilité et documentation technique

Le code est commenté et structuré en modules clairs (NLP, matching, interface).
Une documentation technique interne permet à un nouvel utilisateur ou développeur de :

- comprendre l'architecture logicielle,
- relancer le projet sur une autre machine,

- adapter certains blocs sans devoir tout reconstruire.

Le système est donc maintenable dans le temps, ce qui évite l'obsolescence prématurée.

Réutilisabilité du code et modularité

Les blocs développés sont réutilisables individuellement :

- le moteur de matching peut être réutilisé dans un autre projet médical,
- le bloc NLP peut être branché sur une autre interface.

Cette logique favorise la mutualisation des développements, au lieu de reconstruire de nouveaux systèmes à chaque fois (réduction de l'empreinte logicielle globale).

Conformité RGPD et données sensibles

Bien que le système ne traite ici aucune donnée réelle, il a été pensé pour ne stocker aucun document médical, ni conserver de logs utilisateurs.

Dans une optique de futur déploiement, il est prévu que :

- les textes soient traités en mémoire uniquement,
- aucune information patient identifiable ne soit conservée,
- une option de suppression instantanée soit offerte à l'utilisateur.

8.5. Zones d'ombre et améliorations potentielles

Malgré les efforts d'optimisation et les choix responsables déjà intégrés au projet, certaines limites environnementales restent à prendre en compte, notamment dans une perspective de déploiement à grande échelle ou de montée en charge. Il s'agit ici d'identifier les points d'attention et de proposer des axes concrets d'amélioration.

8.5.1. Limitations actuelles

Utilisation de modèles pré-entraînés volumineux

Le modèle BioLORD, bien que pertinent pour le domaine médical, reste un modèle de type transformer lourd en termes de ressources mémoire et de calcul. Son utilisation sur des machines peu puissantes (sans GPU, ou en environnement hospitalier contraint) peut entraîner une surconsommation ou une lenteur notable.

Absence de profiling énergétique précis

À ce stade, nous ne disposons pas de mesures quantitatives précises de l'empreinte énergétique (CPU/GPU usage, RAM, cycles de calcul). Cela limite la capacité à quantifier objectivement l'impact du système, même si qualitativement il reste modéré.

Pas encore de gestion adaptative des ressources

Le système traite tous les textes de manière identique, quel que soit leur contenu ou leur longueur. Il n'y a pas encore de logique de détection anticipée de complexité ou de déclenchement conditionnel des modèles (par ex. : n'utiliser BioLORD que si les règles ne suffisent pas).

8.5.2. Améliorations possibles à moyen terme

Intégration d'un mode "basse consommation"

Ajouter une option dans l'interface permettant à l'utilisateur de choisir entre :

- un mode rapide (règles simples, classification légère),
- un mode complet (analyse sémantique par modèle transformer).

Mesure de l'impact via outils type CodeCarbon

Utiliser une bibliothèque comme CodeCarbon permettrait de suivre, pendant les tests, la consommation énergétique réelle du projet, et de fixer des objectifs de réduction.

Utilisation de modèles allégés ou distillés

En remplaçant BioLORD par une version distillée ou par un modèle plus compact (ex. : MiniLM ou DistilBERT médical), il serait possible de diviser la consommation par 2 ou 3 tout en gardant une performance proche.

Formation à l'éco-conception pour les futurs contributeurs

Documenter les bonnes pratiques directement dans le dépôt Git ou dans la documentation projet : évite la dérive logicielle si le système est repris par d'autres équipes ou industrialisé.

9. Perspectives et améliorations futures

9.1. Ouvertures techniques à court et moyen terme

Le projet AIME, dans sa version actuelle, constitue un prototype fonctionnel qui répond aux objectifs pédagogiques de l'UE et aux besoins de base d'un codage médical semi-automatisé en psychiatrie. Cependant, plusieurs axes d'amélioration et d'extension ont été identifiés au fil du développement, ouvrant la voie à une version plus robuste, plus fine et plus adaptée à un usage réel en milieu hospitalier.

9.1.1. Enrichissement sémantique des entités médicales

Actuellement, le système repose principalement sur les entités extraites par un modèle NER biomédical (DrBERT-CASM2) combiné à des règles. Il pourrait être intéressant de :

- Ajouter un moteur de contextualisation, capable de tenir compte des relations entre entités (antécédents, comorbidités, chronologie).
- Intégrer une couche de désambiguïsation sémantique, par exemple via l'analyse des dépendances syntaxiques ou l'usage de modèles de type T5/UMLS.

9.1.2. Meilleure gestion des temporalités et du contexte

L'un des points faibles du système reste la difficulté à distinguer les faits passés, actuels ou probables.

Cela pourrait être amélioré par :

- Un module de détection des indices temporels ("en 2015", "ancien", "récidive"),
- Une classification des entités selon leur actualité clinique (active, résolue, hypothétique).

9.1.3. Passage à un traitement multilingue

Dans une perspective de généralisation à d'autres contextes ou pays, le système pourrait être étendu au bilinguisme français/anglais, voire à d'autres langues utilisées dans les environnements médicaux. Cela nécessiterait :

- L'utilisation de modèles multilingues ou traduits (ex. : mBERT, XLM-R),
- L'adaptation de la base CIM-10 dans différentes langues,
- Un module de traduction médicale spécifique, afin de préserver la fidélité terminologique.

9.1.4. Intégration avec des systèmes hospitaliers existants

À terme, une interface plus évoluée pourrait permettre une intégration directe dans un Dossier Patient Informatisé (DPI). Cela supposerait :

- Un packaging du système sous forme d'API ou de microservice,
- Une sécurisation des flux (RGPD, chiffrement),
- Un accès aux comptes rendus via des connecteurs HL7 ou FHIR.

9.2. Perspectives d'industrialisation et déploiement clinique

Si le projet AIME devait quitter le cadre académique pour s'inscrire dans une logique de déploiement réel, notamment au sein d'établissements de santé comme les Hôpitaux d'Instruction des Armées (HIA), plusieurs adaptations techniques, réglementaires et organisationnelles devraient être envisagées.

9.2.1. Mise en conformité réglementaire

Tout système manipulant des données médicales réelles doit impérativement :

- Respecter le Règlement Général sur la Protection des Données (RGPD) : traitement local, anonymisation, consentement, droit à l'effacement, etc.
- Être conforme à la loi française sur les systèmes d'aide à la décision médicale (SADM), en particulier si les codes proposés sont intégrés dans des circuits de facturation ou de diagnostic.
- Disposer d'une traçabilité complète : chaque décision algorithmique devrait pouvoir être auditée, expliquée, et révisée par un professionnel de santé.*

9.2.2. Interface utilisateur avancée et ergonomie clinique

Pour répondre aux exigences d'un praticien en contexte hospitalier, l'interface devrait :

- Gérer l'import direct de documents médicaux standardisés (.docx, HL7, FHIR...),
- Permettre une validation rapide, modification ou suppression des codes proposés,
- Afficher une aide contextuelle (définitions CIM-10, synonymes, lien avec l'historique du patient),
- S'intégrer dans un workflow existant : bouton intégré dans un DPI, suggestion passive, etc.

9.2.3. Formation, acceptabilité et accompagnement au changement

Aucun outil IA, aussi performant soit-il, ne peut être adopté sans :

- Une formation des utilisateurs finaux (praticiens, secrétaires médicales, chefs de service),
- Une période d'expérimentation encadrée,
- La mise en place d'un comité de validation clinique, pour analyser les cas litigieux, affiner les règles métier, et bâtir la confiance dans le système.

9.2.4. Industrialisation logicielle

Sur le plan technique, une industrialisation nécessiterait :

- La mise en place d'une architecture modulaire API-compatible, déployable sur serveur sécurisé,
- L'ajout de tests automatisés (CI/CD), logs, supervision et alertes,
- Une gestion centralisée des modèles (mise à jour, versionning, journalisation des décisions),
- Une compatibilité multiplateforme (web, intranet hospitalier, application mobile)

9.3. Évolutions à long terme

Le projet AIME peut aussi constituer une brique fondatrice pour un système d'IA plus vaste, intégrant d'autres volets cliniques :

- Codage automatique multi-spécialités (cardiologie, gériatrie, pédiatrie…),
- Analyse multimodale (croisement texte + imagerie médicale + données biologiques),
- Détection d'erreurs de codage ou systèmes d'alerte prédictive en lien avec le PMSI,
- Participation à un réseau d'apprentissage fédéré, où chaque hôpital enrichit un modèle global tout en gardant ses données en local (confidentialité assurée).

Conclusion intermédiaire :

AIME, dans sa forme actuelle, est un prototype académique prometteur. Mais avec les bons choix techniques, humains et réglementaires, il a le potentiel de devenir un outil clinique fiable, évolutif et structurant, capable d'aider concrètement les professionnels de santé dans leur pratique quotidienne.

10. Conclusion

Le projet AIME s'est donné pour ambition de répondre à un enjeu central du monde médical : automatiser le codage CIM-10 à partir de comptes rendus cliniques, tout en conservant la rigueur et la traçabilité exigées dans un contexte hospitalier. En s'appuyant sur des techniques modernes de traitement du langage naturel, des modèles de similarité sémantique, et une interface simple d'utilisation, notre équipe a su concevoir un prototype fonctionnel, cohérent et modulaire.

Au fil de ce projet, nous avons appris à :

- gérer un système complexe par blocs indépendants et intégrables,
- adopter une démarche d'ingénierie rigoureuse (tests, validation, documentation),
- prendre en compte les réalités métiers des soignants, bien au-delà des enjeux purement algorithmiques,
- mais aussi à réfléchir aux impacts éthiques et environnementaux de nos choix techniques.

Le système final, bien qu'encore perfectible, démontre une capacité concrète à extraire des informations pertinentes, à proposer des codes pertinents avec un bon taux de réussite, et à laisser la décision finale à l'utilisateur humain.

Mais surtout, AIME ouvre la voie à une IA médicale utile, responsable, et déployable. Loin d'automatiser le jugement clinique, elle vient en appui du praticien, comme un coéquipier numérique, discret mais efficace.

En somme, ce projet nous a permis de croiser deux mondes – celui de l'ingénierie logicielle et celui de la médecine – et de faire le pari que, bien encadrée, l'intelligence artificielle peut réellement contribuer à améliorer la qualité des soins tout en allégeant la charge des soignants.

Références

[1] Nabla Technologies. Nabla - intelligence artificielle pour la santé. https://www.nabla.com/fr/, 2024. Consulté le 10 décembre 2024. 6

[2] Amazon Web Services (AWS). Healthscribe - services d'intelligence artificielle pour la santé. https://aws.amazon.com/fr/healthscribe/, 2024. Consulté le 10 décembre 2024. 6

[3] Nuance Communications. Dragon ambient experience (dax) copilot - solutions cliniques ia. https://www.nuance.com/healthcare/dragon-ai-clinical-solutions/dax-copilot.html, 2024. Consulté le 10 décembre 2024. 6

[4] S. El Khoury, C. A. Mikael, S. El Helou, T. Yaacoub, E. Mikhael, Y. Abou Boutros, A. Mansour, and C. Yaghi. AI-Driven Automation of Medical Keyword Extraction and ICD10 Code Association in Gastroenterology Records. 2024. 6